

Piloto Automático

Nome do arquivo: “piloto.x”, onde x deve ser c, cpp, pas, java, js, py2 ou py3

Uma grande fábrica de carros elétricos está realizando melhorias no sistema de piloto automático e precisa da sua ajuda para implementar um programa que decida se um carro B, que está trafegando no meio de dois carros A e C, precisa acelerar, desacelerar ou manter a velocidade atual. Os carros são iguais e os sensores do piloto automático vão fornecer, como entrada, a posição atual da traseira dos três carros. Veja um exemplo na figura.



O carro B precisa ser acelerado se a distância da sua traseira para a traseira do carro A for menor do que a distância da sua traseira para a traseira do carro C. Se for maior, ele precisa ser desacelerado. Se for igual, precisa manter a velocidade atual. Quer dizer, o carro B precisa ser acelerado se $(B - A) < (C - B)$, desacelerado se $(B - A) > (C - B)$ e manter a velocidade se $(B - A)$ for igual a $(C - B)$.

Entrada

A primeira linha da entrada contém um inteiro A . A segunda linha da entrada contém um inteiro B . A terceira linha da entrada contém um inteiro C . Os três inteiros representam as posições atuais das traseiras dos carros A, B e C, respectivamente.

Saída

Seu programa deve imprimir uma linha contendo um inteiro: 1 se o carro B precisa acelerar; -1 se precisa desacelerar; ou 0 se precisa manter a velocidade atual.

Restrições

- $0 \leq A < B < C \leq 500$

Exemplos

Exemplo de entrada 1 10 23 38	Exemplo de saída 1 1
Exemplo de entrada 2 105 212 319	Exemplo de saída 2 0
Exemplo de entrada 3 80 120 132	Exemplo de saída 3 -1

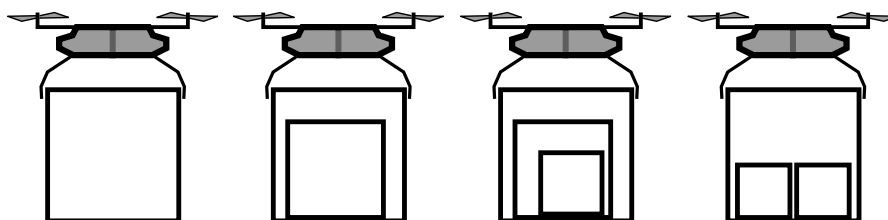
Entrega de Caixas

Nome do arquivo: “caixas.x”, onde x deve ser c, cpp, pas, java, js, py2 ou py3

Você precisa transportar três caixas vazias usando um drone que pode levantar uma caixa por vez apenas em cada viagem. Quer dizer, sempre dá para transportar as três caixas vazias fazendo três viagens do drone. Mas talvez dê para fazer menos do que três viagens, se for possível colocar uma caixa dentro de outra. As caixas têm formato de cubo e a única restrição para uma caixa ser colocada dentro de outra é o tamanho, não importando o peso.

Uma caixa de tamanho X pode ser colocada dentro de uma caixa de tamanho Y se $X < Y$. Note, portanto, que uma caixa não cabe dentro de outra do mesmo tamanho. Além disso, duas caixas de tamanhos X e Y podem ser colocadas, lado a lado, dentro de uma caixa de tamanho Z se $(X + Y) < Z$.

A figura ilustra as quatro configurações possíveis para o drone fazer uma viagem.



Neste problema, os tamanhos das três caixas são dados em ordem crescente e seu programa deve computar o número mínimo de viagens que o drone pode fazer para transportar todas as três caixas.

Entrada

A primeira linha da entrada contém um inteiro A . A segunda linha da entrada contém um inteiro B . A terceira linha da entrada contém um inteiro C . Os três inteiros representam os tamanhos das três caixas.

Saída

Seu programa deve imprimir uma linha contendo um inteiro, representando o número mínimo de viagens que o drone pode fazer para transportar todas as três caixas.

Restrições

- $1 \leq A \leq B \leq C \leq 1000$

Exemplos

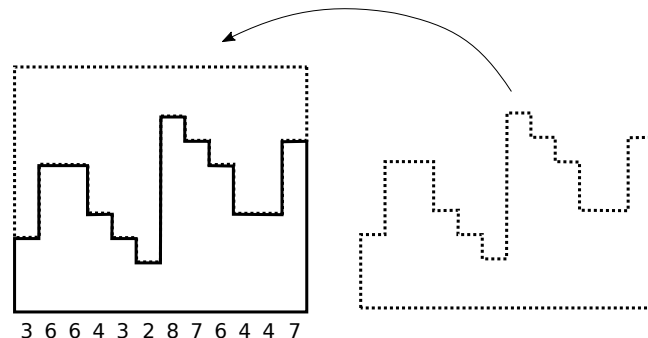
Exemplo de entrada 1 12 45 188	Exemplo de saída 1 1
Exemplo de entrada 2 67 67 67	Exemplo de saída 2 3

Exemplo de entrada 3 111 463 463	Exemplo de saída 3 2
Exemplo de entrada 4 72 72 345	Exemplo de saída 4 1
Exemplo de entrada 5 30 30 55	Exemplo de saída 5 2

Escher

Nome do arquivo: “**escher.x**”, onde **x** deve ser **c**, **cpp**, **pas**, **java**, **js**, **py2** ou **py3**

M. C. Escher foi um artista gráfico holandês que fazia incríveis ilustrações onde preenchia a tela com objetos auto-similares, cujos contornos encaixam neles próprios, criando simetrias geométricas muito impressionantes. Veja um exemplo dessa ideia na figura, que mostra um objeto que é um perfil ortogonal definido por uma sequência de números naturais representando a sequência de alturas. Podemos pegar uma cópia do objeto, rotacionar 180 graus e encaixar perfeitamente no objeto original, formando um retângulo.



Em termos mais gerais, se uma sequência de N números naturais representando a sequência de alturas for $A_1, A_2, A_3, \dots, A_{N-2}, A_{N-1}, A_N$, o perfil definido será chamado de perfil Escher se tivermos $A_1 + A_N$ igual a $A_2 + A_{N-1}$ igual a $A_3 + A_{N-2}$, e assim por diante. Neste problema, será dada a sequência de alturas que definem o perfil e seu programa deve decidir se o perfil é Escher, ou não.

Entrada

A primeira linha da entrada contém um número N , indicando quantos números tem a sequência. A segunda linha da entrada contém N números naturais, A_i , para $1 \leq i \leq N$, definindo a sequência de alturas do perfil.

Saída

Seu programa deve imprimir uma linha contendo o caractere **S**, se o perfil for Escher; ou **N**, senão.

Restrições

- $3 \leq N \leq 10000$.
- $1 \leq A_i \leq 1000$, para todo $1 \leq i \leq N$.

Informações sobre a pontuação

- Em um conjunto de casos de teste somando 20 pontos, $N = 3$.
- Em um conjunto de casos de teste somando 80 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
12 3 6 6 4 3 2 8 7 6 4 4 7	S

Exemplo de entrada 2	Exemplo de saída 2
5 2 1 9 13 12	N

Fissura Perigosa

Nome do arquivo: “fissura.x”, onde x deve ser c, cpp, pas, java, js, py2 ou py3

A erupção do vulcão Kilauea em 2018 no Havaí atraiu a atenção de todo o mundo. Inicialmente a força da erupção era menor e a lava avançou para o sul com relativamente poucos danos. Após algumas semanas, porém, a fissura 8 começou a jorrar com mais força e a lava avançou também para o norte trazendo muita destruição.

Você está ajudando na implementação de um sistema para simular a área por onde a lava avançaria, em função da força da erupção. O mapa será representado simplificadaamente por uma matriz quadrada de caracteres, de 1 a 9, indicando a altitude do terreno em cada posição da matriz. Vamos considerar que a fissura 8, por onde a erupção se inicia, está sempre na posição do canto superior esquerdo da matriz. Dada a força da erupção, que será um valor inteiro, de 0 a 9, seu programa deve imprimir a matriz de caracteres representando o avanço final da lava. Se a lava consegue invadir uma posição da matriz, o caractere naquela posição deve ser trocado por um asterisco (*). Uma posição será invadida pela lava se seu valor for menor ou igual à força da erupção e

- for a posição inicial; ou
- estiver adjacente, ortogonalmente (abaixo, acima, à esquerda ou à direita), a uma posição invadida.

A figura abaixo mostra um exemplo de mapa e o avanço final da lava para quatro forças de erupção: 1, 3, 6 e 8, respectivamente da esquerda para a direita.

27755478	*7755478	*7755478	*****
29985439	*9985439	*9985439	*99*****
34899989	*4899989	**899989	***999*9
22115569	****5569	*****9	*****9
66736689	667*6689	**7***89	*****9
99886555	99886555	9988****	99*****
44433399	44433399	*****99	*****99
99986991	99986991	9998*991	999**991

Entrada

A primeira linha da entrada contém dois inteiros N e F representando, respectivamente o número de linhas (que é igual ao de colunas) da matriz e a força da erupção. Cada uma das N linhas seguintes contém uma string de N caracteres, entre 1 e 9, indicando o mapa de entrada.

Saída

Seu programa deve imprimir N linhas contendo, cada uma, N caracteres representando o avanço final da lava de acordo com o enunciado.

Restrições

- $1 \leq N \leq 500$
- $0 \leq F \leq 9$

Informações sobre a pontuação

- Em um conjunto de casos de teste somando 20 pontos, $N \leq 10$.
- Em um conjunto de casos de teste somando 20 pontos, $10 < N \leq 100$.
- Em um conjunto de casos de teste somando 60 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1 8 6 27755478 29985439 34899989 22115569 66736689 99886555 44433399 99986991	Exemplo de saída 1 *7755478 *9985439 **899989 *****9 **7***89 9988**** *****99 9998*991
Exemplo de entrada 2 5 4 25679 35234 17182 39993 11223	Exemplo de saída 2 *5679 *5*** *7*8* *999* *****
Exemplo de entrada 3 2 8 91 11	Exemplo de saída 3 91 11

Pandemia

Nome do arquivo: “**pandemia.x**”, onde **x** deve ser **c**, **cpp**, **pas**, **java**, **js**, **py2** ou **py3**

Um grupo de amigos, preocupados por ter que prestar o ENEM este ano, resolveu iniciar o ano fazendo reuniões de estudo. Mas eles não esperavam que uma epidemia com um novo vírus ocorresse na região em que moravam. Nessa epidemia específica, os sintomas da doença aparecem muitos dias depois do contágio, mas mesmo sem sintomas uma pessoa infectada infecta todos com quem tenha o mínimo contato.

O grupo de amigos também não sabia que um deles havia sido infectado, sem saber, por pessoas de fora do grupo, o que fez a infecção se espalhar pelos amigos do grupo. Felizmente todos os amigos infectados se recuperaram e passam bem.

Muitas reuniões de estudo aconteceram, mas nem todos os amigos participaram de todas as reuniões.

Você receberá a informação de quais amigos participaram de cada reunião. Além disso, você receberá também a informação de qual amigo participou de reunião do grupo após ter sido infectado por pessoas de fora do grupo, e em qual reunião isso ocorreu. Você deve assumir que:

1. todos os amigos que participaram de reunião em que ao menos um deles estava infectado também foram infectados.
2. o único amigo infectado por pessoas de fora do grupo é o que foi informado. No caso de todos os outros amigos que foram infectados a infecção aconteceu em reunião do grupo.

Escreva um programa para determinar quantos amigos, ao final da sequência de reuniões, foram infectados.

Entrada

A primeira linha da entrada contém dois números inteiros N , M , respectivamente o total de amigos do grupo e o total de dias em que houve reunião. Os amigos são identificados por números inteiros de 1 a N , as reuniões são identificadas por números inteiros de 1 (primeira reunião) a M (última reunião). A segunda linha contém dois números inteiros I e R , respectivamente o identificador do amigo que foi infectado por pessoas de fora do grupo e o número da primeira reunião em que ele participou infectado. Cada uma das M linhas seguintes contém a informação dos participantes de uma reunião, em sequência; ou seja, a primeira linha descreve os participantes da reunião 1, a segunda linha descreve os participantes da reunião 2 e assim por diante. Cada uma dessas linhas inicia com um número A , o total de amigos que participaram dessa reunião, seguido de A inteiros P_i identificando cada amigo participante da reunião.

Saída

Seu programa deve produzir um inteiro representando o número total de amigos infectados ao final do mês.

Restrições

- $2 \leq N \leq 1000$
- $2 \leq M \leq 1000$
- $1 \leq I \leq N$
- $1 \leq R \leq M$
- $1 \leq A \leq N$
- $1 \leq P_i \leq N$ para $1 \leq i \leq A$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 20 pontos, $N \leq 10$ e $M \leq 10$.
- Para um conjunto de casos de testes valendo 60 pontos, $10 < N \leq 500$ e $10 < M \leq 500$.
- Para um conjunto de casos de testes valendo 20 pontos, nenhuma restrição adicional.

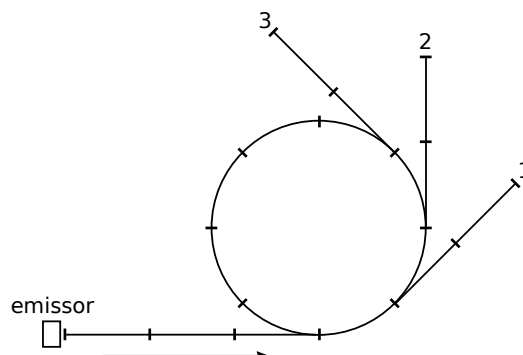
Exemplos

Exemplo de entrada 1 4 3 2 1 2 1 2 3 3 1 2 2 2 1	Exemplo de saída 1 3
Exemplo de entrada 2 5 6 3 4 2 1 3 4 1 2 3 5 2 1 3 2 1 3 2 4 5 2 2 4	Exemplo de saída 2 2
Exemplo de entrada 3 10 5 2 1 6 7 5 1 9 6 2 3 9 4 6 3 2 9 5 3 8 5 7 2 8 9	Exemplo de saída 3 8

Acelerador de Partículas

Nome do arquivo: “`acelerador.x`”, onde `x` deve ser `c`, `cpp`, `pas`, `java`, `js`, `py2` ou `py3`

A universidade está inaugurando um grande acelerador de partículas, com um emissor e três sensores, numerados 1, 2 e 3. Uma partícula, após sair do emissor, entra no acelerador onde pode dar várias voltas sendo acelerada a velocidades muito altas. Num determinado momento, a partícula sai do acelerador por uma das três saídas, atingindo um dos sensores. A figura mostra o caminho por onde as partículas trafegam, com uma graduação de 1 quilômetro. Por exemplo, do emissor até o acelerador são 3 quilômetros e a circunferência do acelerador tem 8 quilômetros.



Neste problema, será dada a distância total, em quilômetros, percorrida por uma certa partícula trafegando do emissor até algum sensor e seu programa deve determinar qual sensor foi atingido pela partícula. Por exemplo, veja que se a distância total for 23 quilômetros, então a partícula tem que ter atingido o sensor 2.

Entrada

A entrada consiste de apenas uma linha contendo um inteiro D , representando a distância total percorrida pela partícula.

Saída

Seu programa deve imprimir uma linha contendo um inteiro, representando o número do sensor que a partícula atingiu.

Restrições

- $6 \leq D \leq 800008$. D sempre será a distância total percorrida entre o emissor e algum sensor.

Exemplos

Exemplo de entrada 1 23	Exemplo de saída 1 2
Exemplo de entrada 2 6	Exemplo de saída 2 1
Exemplo de entrada 3 9192	Exemplo de saída 3 3

Promoção de Primeira

Nome do arquivo: “promocao.x”, onde x deve ser `c`, `cpp`, `pas`, `java`, `js`, `py2` ou `py3`

O reino da Linearlândia possui N cidades espalhadas por seu vasto território, sendo que $N - 1$ pares distintos de cidades estão ligados diretamente por uma rodovia bi-direcional. Esses pares foram escolhidos de forma que existe exatamente um caminho entre qualquer par de cidades, possivelmente passando por outras cidades no meio do caminho. Cada rodovia da Linearlândia é servida por uma linha de ônibus, que faz viagens de ida e volta entre as duas cidades, operada por apenas uma empresa, como manda a lei determinada pelo Rei. O problema é que existem apenas duas empresas de ônibus: a RoyalBus e a ImperialBus.

Cada viagem entre duas cidades ligadas diretamente por uma rodovia custa uma passagem da empresa que opera aquela linha. Ao chegar numa cidade, se o passageiro quiser prosseguir viagem para outra cidade, ele tem que desembarcar, entrar em outro ônibus e pagar outra passagem. Só que o Rei determinou, para o feriadão anual de celebração da Linearidade Real, uma estranha promoção: sempre que o passageiro entrar no ônibus de uma empresa ele não precisa pagar a passagem se sua viagem imediatamente anterior foi pela outra empresa. Quer dizer, se o caminho alterna entre a RoyalBus e a ImperialBus, só é preciso pagar uma passagem, a primeira.

Neste problema, dada a descrição da malha de rodovias da Linearlândia, seu programa deve computar o número máximo de cidades num caminho, começando em qualquer cidade, para o qual é preciso pagar apenas uma passagem para ir da cidade inicial até a cidade final do caminho. O número de cidades no caminho inclui a cidade inicial e a cidade final.

Entrada

A primeira linha da entrada contém um inteiro N , representando o número de cidades da Linearlândia. As cidades são numeradas de 1 até N . As $N - 1$ linhas seguintes contêm, cada uma, três inteiros A , B e E , indicando que existe uma rodovia entre as cidades A e B e que a linha de ônibus entre elas é operado pela empresa E (0 para RoyalBus, 1 para ImperialBus).

Saída

Seu programa deve imprimir uma linha contendo um inteiro representando o número máximo de cidades num caminho para o qual é preciso pagar apenas uma passagem durante a celebração da Linearidade Real.

Restrições

- $2 \leq N \leq 50000$
- $1 \leq A \leq N$
- $1 \leq B \leq N$
- $0 \leq E \leq 1$

Informações sobre a pontuação

- Em um conjunto de casos de teste somando 20 pontos, o número máximo de rodovias chegando em qualquer cidade é dois. Quer dizer, a malha é um longo caminho passando por todas as cidades.
- Em um conjunto de casos de teste somando 20 pontos, $N \leq 10^3$.
- Em um conjunto de casos de teste somando 20 pontos, $10^3 < N \leq 10^4$.
- Em um conjunto de casos de teste somando 40 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1 8 3 1 0 2 7 0 6 8 1 1 4 1 5 4 1 4 7 1 7 6 0	Exemplo de saída 1 4
Exemplo de entrada 2 2 1 2 0	Exemplo de saída 2 2
Exemplo de entrada 3 18 13 16 0 16 15 1 16 12 0 14 12 0 12 8 1 1 18 0 1 3 0 2 3 1 3 8 1 11 17 1 17 10 1 8 17 0 6 7 0 9 7 0 5 7 1 4 5 0 8 5 1	Exemplo de saída 3 6

Bingo!

Nome do arquivo: “bingo.x”, onde x deve ser `c`, `cpp`, `pas`, `java`, `js`, `py2` ou `py3`

O grande prêmio do Bingo de São João será um carro zero-quilômetro. Todo mundo quer ser o primeiro a completar sua cartela, claro. São N cartelas identificadas de 1 até N que contêm, cada uma, K números distintos entre os números naturais de 1 até U , para $K < U$. Um número, claro, pode aparecer em mais de uma cartela e duas cartelas podem até ser iguais, ter o mesmo conjunto de números. Justamente por isso, veja que pode acontecer empate com mais de uma cartela sendo completada no mesmo instante.

Neste problema, serão dados na entrada os conjuntos de números de todas as cartelas e a sequência de números sorteados, que será uma permutação dos naturais de 1 até U . Seu programa deve determinar qual ou quais cartelas vão ser completadas primeiro e ganhar o carro.

Por exemplo, para $N = 4$, $K = 5$ e $U = 10$, com as cartelas dadas pela tabela abaixo, se a sequência de números sorteados for $[7, 3, 5, 2, 6, 1, 9, 10, 4, 8]$, então haverá uma cartela vencedora, a número 3.

número da cartela					
1	3	10	8	7	2
2	4	1	7	10	9
3	9	1	5	3	6
4	6	8	1	5	7

Entrada

A primeira linha da entrada contém três inteiros N , K e U representando respectivamente o número de cartelas, quantos números cada cartela contém e o maior natural que pode ocorrer numa cartela. As N linhas seguintes contêm, cada uma, K inteiros distintos C_i , para $1 \leq i \leq K$, representando o conjunto de números de cada cartela, da cartela 1 até a N . A última linha da entrada contém U inteiros indicando a sequência de números sorteados, uma permutação dos naturais entre 1 e U .

Saída

Seu programa deve imprimir uma linha contendo os números identificadores das cartelas vencedoras do carro, em ordem crescente.

Restrições

- $1 \leq N \leq 1000$
- $1 \leq K \leq 1000$
- $1 \leq U \leq 10000$

Informações sobre a pontuação

- Em um conjunto de casos de teste somando 15 pontos, $N \leq 100$, $K \leq 50$, $U \leq 100$, e apenas uma cartela é vencedora.
- Em um conjunto de casos de teste somando 15 pontos, $N \leq 100$, $K \leq 50$, $U \leq 100$.
- Em um conjunto de casos de teste somando 30 pontos, $N \leq 100$, $K \leq 500$, $U \leq 1000$.
- Em um conjunto de casos de teste somando 40 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1 4 5 10 3 10 8 7 2 4 1 7 10 9 9 1 5 3 6 6 8 1 5 7 7 3 5 6 1 9 2 10 4 8	Exemplo de saída 1 3
Exemplo de entrada 2 5 4 9 1 9 3 2 4 5 6 7 2 3 5 4 2 6 8 1 2 5 7 9 1 9 7 4 5 3 2 8 6	Exemplo de saída 2 1 3 5

Paciente Zero

Nome do arquivo: “paciente.x”, onde x deve ser `c`, `cpp`, `pas`, `java`, `js`, `py2` ou `py3`

Quando ocorre uma epidemia por uma nova espécie de vírus, uma das tarefas dos infectologistas é determinar o *Paciente Zero*, ou seja, a pessoa que foi infectada primeiro pelo novo vírus. A primeira infecção é geralmente causada por transmissão do novo vírus de algum hospedeiro não humano como morcego, rato, ou macaco, para um humano. Esse primeiro humano infectado então infecta outras pessoas, que por sua vez infectam ainda outras pessoas, e assim surge a epidemia.

A procura pelo Paciente Zero é feita através de um minucioso trabalho de entrevistas com infectados para determinar quem contaminou quem desde o início da epidemia. Vamos chamar de “cadeia de infecção” a sequência de infecção causada por uma pessoa. Por exemplo, se as entrevistas determinarem que João infectou Clarisse, Clarisse infectou Silvia, e Silvia infectou Rafael, a cadeia de infecção de João é Clarisse $\rightarrow$ Silvia $\rightarrow$ Rafael. O Paciente Zero é definido como a pessoa infectada com o vírus que não foi infectada por nenhum outro humano, ou seja, não faz parte da cadeia de infecção de nenhuma outra pessoa. Às vezes não é possível determinar um único Paciente Zero, e nesse caso um grupo de pessoas são designadas como Pacientes Zero.

Nesta tarefa você deve determinar o Paciente ou Pacientes Zero a partir das cadeias de infecção determinadas pelos infectologistas.

Entrada

A primeira linha da entrada contém dois números inteiros N e C , respectivamente o total de pessoas infectadas e o número de cadeias de transmissão. As pessoas são identificadas por números inteiros de 1 a N . Cada uma das C linhas seguintes contém a informação sobre uma cadeia de transmissão. A linha inicia com um número P , o identificador da pessoa infectante, seguido de um número I , o total pessoas nessa cadeia de transmissão, seguido de I inteiros X_i identificando as pessoas na cadeia de transmissão.

Cada pessoa consta, no máximo, de uma cadeia de transmissão. Em outras palavras, se um pessoa aparece uma cadeia de transmissão, ela não aparece em nenhuma outra cadeia de transmissão.

Saída

Se houver apenas um Paciente Zero, seu programa deve produzir na saída uma linha contendo o número identificador do Paciente Zero. Se houver K Pacientes Zero, seu programa deve produzir na saída K linhas, cada uma contendo o identificador um Paciente Zero distinto, em ordem crescente do número de identificação dos pacientes.

Restrições

- $2 \leq N \leq 1000$
- $1 \leq C \leq N - 1$
- $1 \leq P \leq N$
- $1 \leq I \leq N - 1$
- $1 \leq X_i \leq N$ para $1 \leq i \leq I$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 20 pontos, $2 \leq N \leq 10$ e há apenas um Paciente Zero.
- Para um conjunto de casos de testes valendo 40 pontos, $10 < N \leq 500$.
- Para um conjunto de casos de testes valendo 40 pontos, nenhuma restrição adicional.

Exemplos

Exemplo de entrada 1 6 2 3 2 4 1 1 3 2 5 6	Exemplo de saída 1 3
Exemplo de entrada 2 6 3 4 2 5 3 2 1 1 5 1 6	Exemplo de saída 2 2 4

Relógio de Atleta

Nome do arquivo: “**relogio.x**”, onde **x** deve ser **c**, **cpp**, **pas**, **java**, **js**, **py2** ou **py3**

Uma empresa está desenvolvendo um novo relógio eletrônico para atletas de alto desempenho. Uma das funcionalidades do novo relógio é que ele vai medir a *frequência cardíaca atual* (batidas do coração por minuto) e a *capacidade de oxigenação atual* do atleta. O relógio também vai calcular e armazenar a *frequência cardíaca em repouso* do atleta.

Os projetistas querem que o relógio emita um aviso para o atleta de que ele ou ela deve:

- *diminuir* o ritmo do exercício se a frequência cardíaca atual é maior do que três vezes a frequência cardíaca em repouso **ou** a capacidade de oxigenação atual é menor do que 95;
- *aumentar* o ritmo do exercício se a frequência cardíaca atual é menor do que duas vezes frequência cardíaca em repouso **e** a capacidade de oxigenação atual é maior do que 97;
- *manter* o ritmo de exercício se nenhuma das condições anteriores ocorrer.

Dadas a frequência cardíaca em repouso, a frequência cardíaca atual e a capacidade de oxigenação atual obtidas pelo relógio, escreva um programa que produza a sugestão que o relógio deve emitir.

Entrada

A entrada é composta por três linhas, cada uma contendo um único número inteiro. A primeira linha contém o inteiro R , a frequência cardíaca em repouso do atleta. A segunda linha contém o inteiro F , a frequência cardíaca atual do atleta. A terceira linha contém o inteiro C , a capacidade de oxigenação atual do atleta.

Saída

Seu programa deve produzir uma única linha, contendo uma única palavra, em letras minúsculas, que deve ser ‘aumentar’, ‘diminuir’, ou ‘manter’, de acordo com os critérios estabelecidos acima.

Restrições

- $35 \leq R \leq 100$
- $35 \leq F \leq 200$
- $50 \leq C \leq 100$

Exemplo de entrada 1 60 119 98	Exemplo de saída 1 aumentar
Exemplo de entrada 2 60 190 100	Exemplo de saída 2 diminuir
Exemplo de entrada 3 50 100 95	Exemplo de saída 3 manter

Exemplo de entrada 4	Exemplo de saída 4
50 100 94	diminuir

Divisão do Tesouro

Nome do arquivo: “**tesouro.x**”, onde **x** deve ser **c**, **cpp**, **pas**, **java**, **js**, **py2** ou **py3**

O Capitão Olho Roxo e seus marinheiros encontraram uma arca com uma grande quantidade de moedas de ouro idênticas. Para a divisão das moedas, todos concordaram com a seguinte sugestão do Capitão:

- cada marinheiro exceto o Capitão deveria receber exatamente o mesmo número de moedas; e
- o Capitão deveria receber o dobro de moedas que um marinheiro recebe.

Pode ser que o fato de o Capitão ser o único com uma pistola a bordo tenha contribuído para a concordância de todos, mas também contribuiu o fato de que na forma proposta a divisão era perfeita, não sobrando ou faltando moedas.

Dados o número de moedas na arca e o número de marinheiros, escreva um programa para determinar quantas moedas o Capitão Olho Roxo recebeu.

Entrada

A primeira linha da entrada contém um número inteiro A , o número de moedas na arca. A segunda linha contém um inteiro N , o número de marinheiros (não contando o Capitão).

Saída

Seu programa deve produzir na saída uma única linha, contendo um único inteiro, o número de moedas que o Capitão Olho Roxo deve receber.

Restrições

- $3 \leq A \leq 10000$
- $1 \leq N \leq 1000$

Exemplo de entrada 1 221 11	Exemplo de saída 1 34
Exemplo de entrada 2 1000 8	Exemplo de saída 2 200
Exemplo de entrada 3 3 1	Exemplo de saída 3 2

Camisetas da Olimpíada

Nome do arquivo: “camisetas.x”, onde x deve ser c, cpp, pas, java, js, py2 ou py3

A Olimpíada Municipal de Programação vai distribuir camisetas para os melhores colocados, e por isso solicitou que os premiados informassem o tamanho preferido da camiseta, entre os tamanhos pequeno e médio.

A empresa que confeccionou as camisetas, por uma falha, pode ter se enganado na quantidade de camisetas para cada tamanho. Foram produzidas camisetas em número suficiente para todos os premiados, mas talvez não do tamanho preferido.

Dadas a lista com os tamanhos preferidos pelos premiados e a quantidade de camisetas de cada tamanho produzidas pela empresa, escreva um programa para determinar se todos os premiados receberão camisetas do tamanho escolhido.

Entrada

A primeira linha contém um inteiro N , o número de premiados. A segunda linha contém N inteiros T_i , indicando os tamanhos solicitados pelos premiados, sendo que $T_i = 1$ representa o tamanho pequeno e $T_i = 2$ representa o tamanho médio. A terceira linha contém um inteiro P , o número de camisetas de tamanho pequeno produzidas. A quarta e última contém um inteiro M , o número de camisetas de tamanho médio produzidas.

Saída

Seu programa deve produzir uma única linha, contendo um único caractere, que deve ser a letra maiúscula ‘S’ se todos os premiados serão atendidos com a camiseta do tamanho que escolheram, ou a letra maiúscula ‘N’ caso contrário.

Restrições

- $1 \leq N \leq 1000$
- $0 \leq P \leq 1000$
- $0 \leq M \leq 1000$
- $N \leq P + M$
- $1 \leq T_i \leq 2$ para $1 \leq i \leq N$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 20 pontos, $1 \leq N \leq 10$.

Exemplo de entrada 1 5 1 1 2 1 2 3 2	Exemplo de saída 1 S
Exemplo de entrada 2 4 2 2 2 2 1 3	Exemplo de saída 2 N

Exemplo de entrada 3	Exemplo de saída 3
6 1 1 1 2 1 1 4 4	N

Três por Dois

Nome do arquivo: “tres.x”, onde x deve ser c, cpp, pas, java, js, py2 ou py3

Uma loja de chocolates está fazendo uma promoção do tipo “leve três pague dois”. Mais precisamente, você pode escolher quaisquer três chocolates da loja e paga apenas pelos dois mais caros que escolher, levando o mais barato gratuitamente. Se você levar mais do que três chocolates, você pode agrupá-los em grupos de três e levar o chocolate mais barato de cada grupo gratuitamente.

Por exemplo, se você escolher chocolates de preços (em reais) 8, 5, 10, 2, 5, 10 e 4, você pode agrupá-los como (8,5,10), (2,5,10) e (4), pagará um total de $(10+8) + (10+5) + 4$, ou seja, 37 reais. Mas você pode agrupá-los de uma forma melhor e assim conseguir pagar menos. Você consegue ver como?

Dados os preços dos chocolates escolhidos, escreva um programa para determinar o menor preço a pagar.

Entrada

A primeira linha da entrada contém um inteiro N , o número de chocolates escolhidos. Cada uma das N linhas seguintes contém um número inteiro P , o preço de um chocolate escolhido.

Saída

Seu programa deve produzir uma única linha, contendo um único número inteiro, o menor preço a pagar pelos chocolates escolhidos.

Restrições

- $1 \leq N \leq 100000$
- $1 \leq P \leq 1000$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 50 pontos, $N \leq 1000$.

Exemplo de entrada 1 4 5 6 6 5	Exemplo de saída 1 17
Exemplo de entrada 2 4 3 3 3 3	Exemplo de saída 2 9

Exemplo de entrada 3 6 7 8 7 7 5 7	Exemplo de saída 3 29
Exemplo de entrada 4 7 8 5 10 2 5 10 4	Exemplo de saída 4 32

Emoticons

Nome do arquivo: “**emoticons.x**”, onde **x** deve ser **c**, **cpp**, **pas**, **java**, **js**, **py2** ou **py3**

Emoticons são símbolos usados para expressar o sentimento de quem escreve uma mensagem. Scott Fahlman, um professor de uma universidade americana, foi o primeiro a propor o uso das sequências de caracteres `: -)` e `: -(`, que viraram respectivamente símbolos para “divertido” e “chateado”.

Vamos definir o *sentimento* expresso em uma mensagem como sendo:

- **neutro**: se o número de símbolos “divertido” é igual ao número de símbolos “chateado”;
- **divertido**: se o número de símbolos “divertido” é maior do que número de símbolos “chateado”;
- **chateado**: se o número de símbolos “chateado” é maior do que número de símbolos “divertido”;

Dada uma mensagem composta por uma cadeia de caracteres, escreva um programa para determinar o sentimento expresso na mensagem.

Entrada

A primeira e única linha da entrada contém a mensagem M como uma cadeia de caracteres.

Saída

Seu programa deve produzir uma única linha, contendo uma única palavra, o sentimento expresso na mensagem da entrada.

Restrições

- $3 \leq$ comprimento de $M \leq 280$
- M é composta por letras não acentuadas, espaços em branco e os caracteres ‘(’ (abre parênteses), ‘)’ (fecha parênteses), ‘:’ (dois-pontos), ‘-’ (hífen), ‘.’ (ponto) e ‘,’ (vírgula).

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 25 pontos, há no máximo um emoticon na mensagem.

Exemplo de entrada 1 Espero que esteja tudo bem :-)	Exemplo de saída 1 divertido
Exemplo de entrada 2 Achei o filme muito divertido.	Exemplo de saída 2 neutro
Exemplo de entrada 3 :-):-(:-(:-)	Exemplo de saída 3 neutro
Exemplo de entrada 4 Sonhei com a prova :-((vou estudar).	Exemplo de saída 4 chateado

Irmãos

Nome do arquivo: “irmaos.x”, onde x deve ser c, cpp, pas, java, js, py2 ou py3

Otávio tem dois irmãos, um mais velho (Orlando) e um mais novo do que ele (Oscar). As idades dos três irmãos formam uma *progressão aritmética*: a diferença de idade dos dois irmãos mais novos (Otávio e Oscar) é igual à diferença de idade dos dois irmãos mais velhos (Orlando e Otávio).

Dadas as idades de Otávio e de seu irmão mais novo, escreva um programa para determinar a idade do irmão mais velho.

Entrada

A primeira linha da entrada contém um inteiro N , a idade do irmão mais novo de Otávio. A segunda linha contém um inteiro M , a idade de Otávio.

Saída

Seu programa deve produzir na saída uma única linha, contendo um único número inteiro, a idade do irmão mais velho de Otávio.

Restrições

- $1 \leq N \leq 40$
- $N \leq M \leq 40$

Exemplo de entrada 1 13 16	Exemplo de saída 1 19
Exemplo de entrada 2 14 14	Exemplo de saída 2 14

Ralouim

Nome do arquivo: “**ralouim.x**”, onde **x** deve ser **c**, **cpp**, **pas**, **java**, **js**, **py2** ou **py3**

Para a tradicional festa infantil de Ralouim, o rei da Nlogônia instalou tendas de distribuição de guloseimas no seu extenso Jardim Real, onde está também situado o Palácio Real.

Cada tenda tem uma quantidade ilimitada de guloseimas. As crianças devem sair do Palácio Real e visitar as tendas para ganhar guloseimas, mas o Rei estabeleceu algumas regras:

- a cada visita a uma tenda, a criança ganha exatamente uma guloseima.
- uma tenda pode ser visitada mais de uma vez pela mesma criança, desde que as visitas não sejam consecutivas (ou seja, uma imediatamente após a outra).
- as distâncias que uma criança percorre para chegar à “próxima tenda” devem ser estritamente decrescentes. Ou seja, a distância que a criança percorre do Palácio até a primeira tenda que a criança visita deve ser maior do que a distância que a criança percorre entre a primeira tenda e a segunda tenda, que por sua vez deve ser maior do que a distância que a criança percorre entre a segunda tenda e a terceira tenda, e assim por diante.

Pedrinho percebeu que se planejar direito suas visitas, pode ganhar muitas guloseimas! Escreva um programa para ajudar Pedrinho a ganhar o maior número possível de guloseimas no Ralouim.

Entrada

A primeira linha da entrada contém um inteiro N , o número de tendas. Cada uma das N linhas seguintes contém dois inteiros X e Y , as coordenadas de uma tenda no Jardim Real. A localização do Palácio Real é $(0,0)$ e não existe tenda com essas coordenadas, todas as tendas têm localizações distintas.

Saída

Seu programa deve produzir uma única linha, contendo um único inteiro, o maior número de guloseimas que Pedrinho pode ganhar.

Restrições

- $1 \leq N \leq 2000$
- $-10000 \leq X \leq 10000$
- $-10000 \leq Y \leq 10000$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 20 pontos, $1 \leq N \leq 50$.
- Para um conjunto de casos de testes valendo 40 pontos adicionais, $1 \leq N \leq 200$.
- Para um conjunto de casos de testes valendo 40 pontos adicionais, nenhuma restrição adicional.

Exemplo de entrada 1	Exemplo de saída 1
4 6 0 5 0 1 0 2 0	5

Exemplo de entrada 2 2 0 3 3 0	Exemplo de saída 2 1
Exemplo de entrada 3 5 6 8 2 1 2 2 2 3 5 9	Exemplo de saída 3 6

Música para Todos

Nome do arquivo: “`musica.x`”, onde `x` deve ser `c`, `cpp`, `pas`, `java`, `js`, `py2` ou `py3`

Uma empresa de *streaming* de músicas lançou uma nova funcionalidade, para grupos de amigos ouvirem em seus aparelhos a mesma música ao mesmo tempo, compartilhando assim um momento de alegria nestes tempos difíceis de pandemia.

Cada amigo no grupo tem que declarar uma música que adora e uma música que detesta. Um amigo fica *satisfeito* se a música que está sendo compartilhada não é a música que ele detesta. Se a música que está sendo compartilhada é detestada por um dos amigos, ele pode trocar a música sendo compartilhada pela música que ele adora. Se há mais de um amigo que detesta a música que está sendo compartilhada, somente o amigo que é cliente há mais tempo da empresa é que pode trocar a música (e troca pela música que adora).

Obviamente, após a troca da música compartilhada pode ser que outro amigo deteste a nova música, e uma nova troca pode ocorrer. E as trocas podem ser intermináveis!

Dadas as preferências de um grupo de amigos e a música sendo compartilhada, e considerando que sempre ocorre troca enquanto houver cliente não satisfeito, escreva um programa para determinar quantas trocas ocorrem até que todos os amigos estejam satisfeitos.

Entrada

A primeira linha da entrada contém três inteiros N , M e C , respectivamente o número de amigos, o número de músicas e a música que está sendo compartilhada. As músicas são identificadas por inteiros de 1 a M . Cada uma das N linhas seguintes descreve as escolhas de um amigo e contém dois inteiros A e D , identificando respectivamente a música adorada e a música detestada. A ordem dos amigos na entrada obedece à ordem em que os amigos se tornaram clientes da empresa (o amigo que aparece antes é cliente há mais tempo).

Saída

Seu programa deve produzir uma única linha na saída, contendo um único inteiro, o número de trocas até que todos os amigos fiquem satisfeitos ou o número -1 se as trocas continuam indefinidamente.

Restrições

- $1 \leq N \leq 10^5$
- $1 \leq M \leq 10^5$
- $1 \leq C \leq M$
- $1 \leq A, D \leq M$ e $A \neq D$

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 40 pontos, $1 \leq N, M \leq 1000$.

Exemplo de entrada 1	Exemplo de saída 1
3 4 2 1 2 2 3 3 2	1

Exemplo de entrada 2 4 5 2 1 3 2 3 3 2 5 1	Exemplo de saída 2 3
Exemplo de entrada 3 3 3 1 1 2 2 3 3 1	Exemplo de saída 3 -1

Garamana

Nome do arquivo: “garamana.x”, onde x deve ser `c`, `cpp`, `pas`, `java`, `js`, `py2` ou `py3`

Um *anagrama* de uma palavra é um rearranjo das letras da palavra. Por exemplo,

1. “rota” é um anagrama de “ator”;
2. “amor” é um anagrama de “roma”; e
3. os anagramas de “aab” são “aab”, “aba” e “baa”.

Um *anagrama curinga* de uma palavra é um anagrama em que algumas das letras podem ter sido substituídas pelo caractere ‘*’ (asterisco). Por exemplo, três possíveis anagramas curingas de “amor” são “*mor”, “a*r” e “r*a”.

Dadas duas palavras, escreva um programa para determinar se a segunda palavra é um anagrama curinga da primeira palavra.

Entrada

A primeira linha da entrada contém P , a primeira palavra. A segunda linha contém A , a segunda palavra.

Saída

Seu programa deve produzir uma única linha, contendo um único caractere, que deve ser ‘S’ se A é um anagrama curinga de P , ou ‘N’ caso contrário.

Restrições

- $1 \leq \text{comprimento de } P \leq 100$
- comprimento de $A = \text{comprimento de } P$
- P é composta por letras minúsculas não acentuadas
- A é composta por letras minúsculas não acentuadas e o caractere ‘*’ (asterisco)

Informações sobre a pontuação

- Para um conjunto de casos de testes valendo 50 pontos, A contém apenas letras minúsculas não acentuadas.

Exemplo de entrada 1 roma ator	Exemplo de saída 1 N
Exemplo de entrada 2 olimpiada poliamida	Exemplo de saída 2 S
Exemplo de entrada 3 microfone *conform*	Exemplo de saída 3 S